

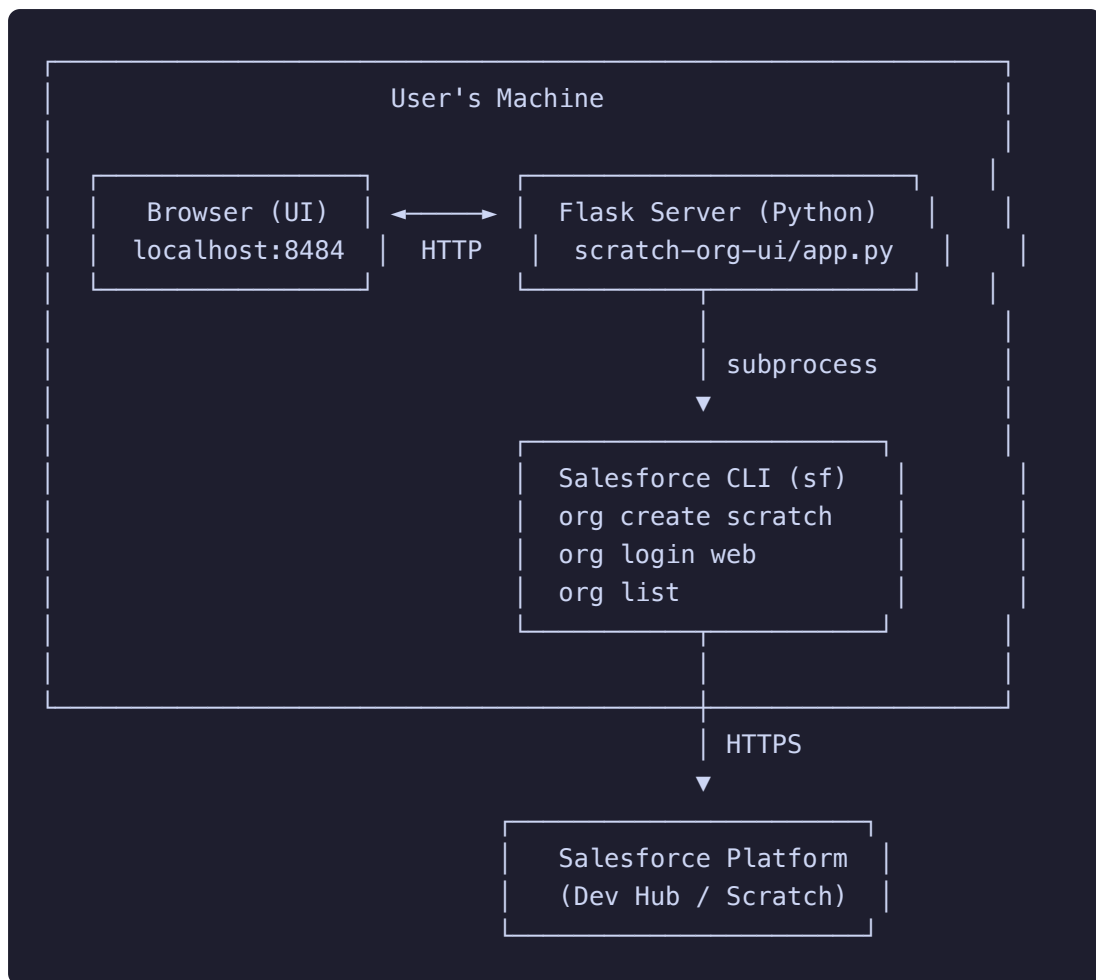
Scratch Org Creator — Technical Documentation

Table of Contents

1. [Architecture Overview](#)
 2. [Project Structure](#)
 3. [File-by-File Breakdown](#)
 4. [Data Flow](#)
 5. [API Endpoints](#)
 6. [Frontend Architecture](#)
 7. [Session Persistence](#)
 8. [Distribution & Packaging](#)
 9. [Security Considerations](#)
 10. [Dependencies](#)
-

Architecture Overview

The Scratch Org Creator is a **local-only** web application that provides an interactive browser UI for creating Salesforce scratch orgs. It follows a client-server architecture running entirely on the user's machine:



Key Design Decisions:

Decision	Rationale
Python Flask	Lightweight, no compilation needed, pre-installed on macOS
Single HTML file	No build step, no npm, instant start
Local vendor directory	Avoid requiring global pip install or admin permissions
Server-Sent Events (SSE)	Real-time streaming of CLI output without WebSocket complexity
Fixed port (8484)	Allows browser localStorage to persist across sessions
Server-side session file	Survives port changes if 8484 is unavailable

Project Structure

```
DevOpsGDC/
├── create-scratch-org.sh           # Main entry point (launcher)
├── build-installer.py             # Generates self-contained dist
├── scratch-org-creator.sh         # Generated single-file distrib
├── README.md                     # User-facing documentation
├── TECHNICAL_DOCUMENTATION.md    # This file
├──
├── scratch-org-ui/               # Web UI application
│   ├── app.py                   # Flask web server (387 lines)
│   ├── features.json            # 296 Salesforce features catalog
│   ├── settings.json            # 25 categories of org settings
│   ├── requirements.txt         # Python dependency declaration
│   ├── .gitignore               # Excludes vendor/, __pycache__
│   ├── templates/
│   │   └── index.html           # Complete single-page UI (1735 lines)
│   ├── vendor/                  # Auto-installed Flask + dependencies
│   ├── session.json             # Current form state (gitignored)
│   └── history.json             # Log of created orgs (gitignored)
└── scratch-orgs-templates/       # Output directory for generated orgs
    ├── my-scratch-org-scratch-def.json
    └── ...
```

File-by-File Breakdown

1. `create-scratch-org.sh` (511 lines)

Purpose: Main entry point. Launches either the web UI (default) or a traditional CLI-based interactive flow.

How it works:

1. **Shell configuration** (lines 1–37): Sets `set -u` (undefined variable errors) and `set -o pipefail`. Defines ANSI color codes and helper functions (`say`, `info`, `ok`, `warn`, `err`, `header`) for formatted terminal output. Detects if stdout is a TTY to conditionally enable colors.
2. **Path setup** (lines 38–46): Resolves `SCRIPT_DIR` using `${BASH_SOURCE[0]}`, sets `REPO_ROOT`, creates the `scratch-orgs-templates/` output directory,

and points `UI_DIR` to `scratch-org-ui/`.

3. **Mode selection** (lines 48–62): Parses `--web`, `--cli`, and `--help` flags. Default mode is `web`.

4. **Web UI launcher** (`launch_web_ui()`, lines 64–137):

5. Checks for `python3` availability
6. Checks if Flask is importable (from system or local `vendor/` directory)
7. If Flask is missing, installs it to `$UI_DIR/vendor` using `pip3 install --target`
8. Attempts to bind to preferred port 8484 (for localStorage persistence), falls back to random port
9. Launches `app.py` as a background process
10. Opens the browser using `open` (macOS), `xdg-open` (Linux), or prints URL
11. Sets up a `trap` for INT/TERM signals to kill the server on Ctrl+C
12. Calls `wait` to keep the script alive until the server exits
13. **CLI mode fallback** (lines 139–511): If web UI fails or `--cli` is passed:
14. Prerequisite checks (sf CLI, python3)
15. Interactive Dev Hub authorization flow
16. Step-by-step prompts for org definition (alias, edition, duration, username, country, features, settings)
17. Builds JSON definition using embedded Python for safe JSON construction
18. Executes `sf org create scratch` with retry logic for invalid email errors
19. Optional password generation
20. Displays org credentials and offers to open in browser

Key technical details: - Uses `printf -v` for variable assignment from prompts (avoids subshell issues) - Embedded Python scripts (heredocs) handle JSON construction to avoid shell quoting issues - The `write_def_file` function uses a full Python script to parse dot-notation settings into nested JSON objects - Error recovery: if `INVALID_EMAIL_ADDRESS` is detected in the create log, it re-prompts and retries

2. `scratch-org-ui/app.py` (387 lines)

Purpose: Flask web server that serves the HTML UI and provides REST API endpoints for all backend operations.

How it works:

1. **Vendor path setup** (lines 19–22): Inserts the local `vendor/` directory into `sys.path` so Flask can be imported without a global install.
2. **Cache control** (lines 30–35): An `@app.after_request` hook adds `Cache-Control: no-cache, no-store, must-revalidate` headers to every response, preventing browser caching issues during development.
3. **Configuration** (lines 37–44): Defines paths to features catalog, settings catalog, history file, and session file — all relative to the script's own directory.
4. **History management** (lines 48–62): `load_history()` reads `history.json`, `save_history_entry()` prepends a new entry and caps at 100 entries.

5. API Routes:

Route	Method	Function
/	GET	Serves <code>index.html</code> template
/api/features	GET	Returns 296 features from <code>features.json</code>
/api/settings	GET	Returns 25 setting categories from <code>settings.json</code>
/api/devhubs	GET	Runs <code>sf org list --json</code> and extracts Dev Hub orgs
/api/authorize	POST	Runs <code>sf org login web</code> for OAuth authorization
/api/history	GET	Returns creation history from <code>history.json</code>
/api/session	GET	Returns saved form state from <code>session.json</code>
/api/session	POST	Saves current form state to <code>session.json</code>
/api/session	DELETE	Removes <code>session.json</code> (after successful creation)
/api/preview	POST	Returns a scratch org definition JSON preview
/api/create	POST	Starts org creation in a background thread
/api/stream/<job_id>	GET	SSE endpoint for real-time output
/api/shutdown	POST	Sends SIGTERM to self for graceful shutdown

1. **Scratch org creation** (`run_create_job` , lines 270–376):
2. Runs in a daemon thread (doesn't block HTTP responses)
3. Writes the definition JSON to `scratch-orgs-templates/`
4. Constructs the `sf org create scratch` command with all parameters
5. Uses `subprocess.Popen` with `stdout=PIPE, stderr=STDOUT` for line-by-line streaming

6. Sets `FORCE_COLOR=0` and `NO_COLOR=1` environment variables to suppress ANSI output from sf CLI
 7. Applies regex `r'\x1b\[([0-9];)*[A-Za-z]|\x1b\[. *?\x07|\r'` to strip any remaining escape sequences
 8. Emits each clean line to a `queue.Queue` consumed by the SSE endpoint
 9. On success: generates password, fetches org details, saves to history
 10. On failure: emits error message with exit code
 11. **SSE streaming** (`stream()`, lines 211–229):
 12. Returns a `text/event-stream` response
 13. Generator function polls the job's queue with 60s timeout
 14. Sends JSON-encoded messages: `{type: "info|output|success|error|warning|details", text: "..."}"`
 15. Sends `{type: "done", status: "success|failed"}` as final message
 16. Sends `{type: "ping"}` every 60s to keep connection alive
-

3. `scratch-org-ui/templates/index.html` (1735 lines)

Purpose: Complete single-page application — HTML, CSS, and JavaScript in one file.

Structure:

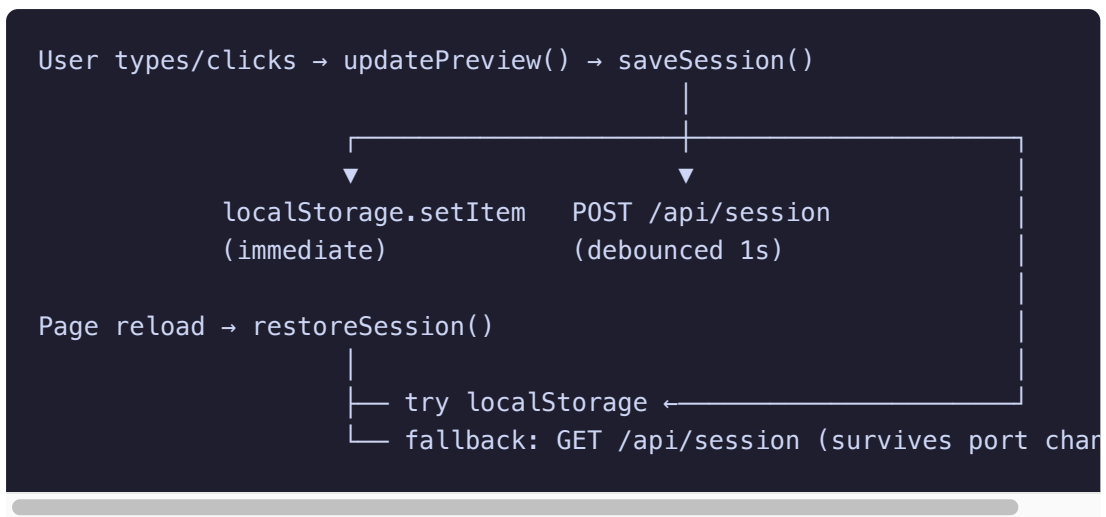
```
index.html
├── <head>
│   ├── Choices.js CSS (CDN)
│   └── <style> (660 lines of CSS)
├── <body>
│   ├── <header> – App title bar
│   ├── <div class="layout"> – CSS Grid (main + sidebar)
│   │   ├── <div class="main-panel">
│   │   │   ├── Step indicator (1–5 progress bar)
│   │   │   ├── Step 1: Dev Hub connection
│   │   │   ├── Step 2: Org definition
│   │   │   ├── Step 3: Features
│   │   │   ├── Step 4: Settings
│   │   │   └── Step 5: Review & Create
│   │   └── <div class="sidebar">
│   │       └── Live JSON preview panel
│   ├── Choices.js library (CDN)
│   └── <script> (800+ lines of JavaScript)
```

CSS Architecture: - CSS custom properties (`:root` variables) for theming: colors, spacing, shadows, fonts - Responsive grid layout (`grid-template-columns: 1fr 380px`) with mobile breakpoint at 1024px - Custom toggle switches using `<label>` + hidden `<input>` + `::before` pseudo-element - Dark theme code panels using Catppuccin-inspired colors - Choices.js override styles for consistent look

JavaScript Architecture:

Function	Purpose
<code>loadDevHubs()</code>	Fetches <code>/api/devhubs</code> , renders clickable cards
<code>authorizeDevHub()</code>	POSTs to <code>/api/authorize</code> , shows loading spinner, refreshes list
<code>loadFeatures()</code>	Fetches <code>/api/features</code> , builds <code><optgroup></code> - organized <code><select></code> , initializes Choices.js
<code>loadSettings()</code>	Fetches <code>/api/settings</code> , builds accordion UI with toggles
<code>updatePreview()</code>	Computes definition JSON from form state, renders syntax-highlighted preview
<code>saveSession()</code>	Debounce (1s) save to both localStorage and <code>/api/session</code>
<code>restoreSession()</code>	Tries localStorage first, falls back to <code>/api/session</code>
<code>createOrg()</code>	POSTs form data to <code>/api/create</code> , opens SSE stream, renders console output
<code>goToStep(n)</code>	Shows/hides step panels, updates progress indicator
<code>filterSettings(query)</code>	Filters settings by label/description/key path
<code>getSettingsFromUI()</code>	Converts toggle states + custom rows into nested JSON object

Session Persistence Flow:



4. `scratch-org-ui/features.json` (1828 lines, 58 KB)

Purpose: Comprehensive catalog of 296 Salesforce scratch org features sourced from official documentation.

Schema:

```
{
  "name": "FeatureName",
  "description": "Human-readable description of what this feature enables",
  "category": "Category Name",
  "requiresValue": false
}
```

For features requiring a numeric value:

```
{
  "name": "FieldService:",
  "description": "Provides the Field Service license.",
  "category": "Field Service",
  "requiresValue": true,
  "valueRange": "1-25"
}
```

Categories (20 total): Analytics & Einstein, Automation, Commerce, Core Platform, Data & Integration, Experience Cloud, Field Service, Industries - Education, Industries - Financial, Industries - Health, Industries - Manufacturing, Industries - Nonprofit, Industries - Public Sector, Marketing, Platform, Sales Cloud, Security & Identity, Service Cloud, Sustainability

How it's used: - Loaded by Flask's `/api/features` endpoint - Frontend groups features into `<optgroup>` elements by category - Choices.js provides searchable multi-select with type-ahead - Features with `requiresValue: true` generate additional numeric input fields

5. `scratch-org-ui/settings.json` (401 lines, 11 KB)

Purpose: Pre-built catalog of 39 common scratch org settings organized into 25 categories with labels, descriptions, and types.

Schema:

```
[
  {
    "category": "Category Name",
    "settings": [
      {
        "key": "dotNotation.pathToSetting",
        "label": "Human-Readable Label",
        "description": "What this setting controls.",
        "type": "boolean",
        "default": true
      }
    ]
  }
]
```

Setting types: - `boolean` — renders as a toggle switch - `string` — renders as a text input field

Categories include: Lightning Experience, Chatter & Collaboration, Enhanced Notes, Path Assistant, Opportunity Settings, Activity Settings, Campaigns, Account Settings, Einstein Features, Security & Access, Email, Knowledge, Case Management, Communities, Forecasting, Territory Management, Contracts, Orders, Quotes, Ideas, Social Profiles, Search, Content Deliveries, Pardot, Custom Address

How the dot-notation key maps to JSON:

```
Key: securitySettings.sessionSettings.forceReLogin
Result JSON:
{
  "settings": {
    "securitySettings": {
      "sessionSettings": {
        "forceReLogin": true
      }
    }
  }
}
```

The `getSettingsFromUI()` JavaScript function splits each key by `.` and builds nested objects.

6. `scratch-org-ui/requirements.txt`

Contents: `flask`

Purpose: Declares the single Python dependency. Used by: - The `pip3 install --target vendor/ flask` command in the launcher script - Reference for anyone wanting to install globally

Transitive dependencies installed: Werkzeug, Jinja2, itsdangerous, click, MarkupSafe, blinker

7. `scratch-org-ui/.gitignore`

```
vendor/
__pycache__/
*.pyc
history.json
session.json
```

Purpose: Prevents committing: - `vendor/` — locally installed Flask and its dependencies (~5 MB) - `__pycache__/` — Python bytecode cache - `history.json` — user-specific creation history (contains org credentials) - `session.json` — in-progress form state

8. `build-installer.py` (232 lines)

Purpose: Generates `scratch-org-creator.sh` — a single self-contained file that embeds all application files as base64-encoded heredocs.

How it works:

1. **Reads source files** (lines 16–21): `app.py`, `features.json`, `settings.json`, `templates/index.html`
2. **Computes content hash** (lines 38–40): SHA-256 of all files concatenated, truncated to 16 hex chars. Used as a version identifier to detect when re-extraction is needed.
3. **Generates shell script** with three sections:
4. **Header:** Prerequisite checks, argument parsing (`--update`, `--help`), `needs_extract()` function that compares stored version hash
5. **Extraction:** For each file, generates a heredoc block: `bash python3 -c "import base64,sys; sys.stdout.buffer.write(base64.b64decode(sys.stdin.read()))" <<'__EOF_APP_PY__' > "$INSTALL_DIR/app.py" <base64-encoded content> __EOF_APP_PY__`
6. **Footer:** Flask installation check, port selection, server launch, browser open
7. **Output** (lines 219–227): Writes to `scratch-org-creator.sh`, sets executable permissions (`chmod 0o755`), prints size and hash.

Extraction strategy: - Uses heredoc with quoted delimiter (`<<'MARKER'`) to prevent shell expansion of base64 content - Pipes through `python3 -c "import base64..."` for decoding - Files are extracted to `~/.scratch-org-creator/` on the user's machine - Re-extraction only happens if version hash changes or files are missing

9. `scratch-org-creator.sh` (179 lines + embedded data, 191 KB)

Purpose: The distributable artifact. A single bash script that contains everything needed to run the tool.

Runtime behavior:

First run:

1. Check python3 exists → error if not
2. Check sf CLI exists → error if not
3. Extract 4 files from embedded base64 to ~/.scratch-org-creator/
4. Install Flask to ~/.scratch-org-creator/vendor/
5. Start server on port 8484
6. Open browser

Subsequent runs:

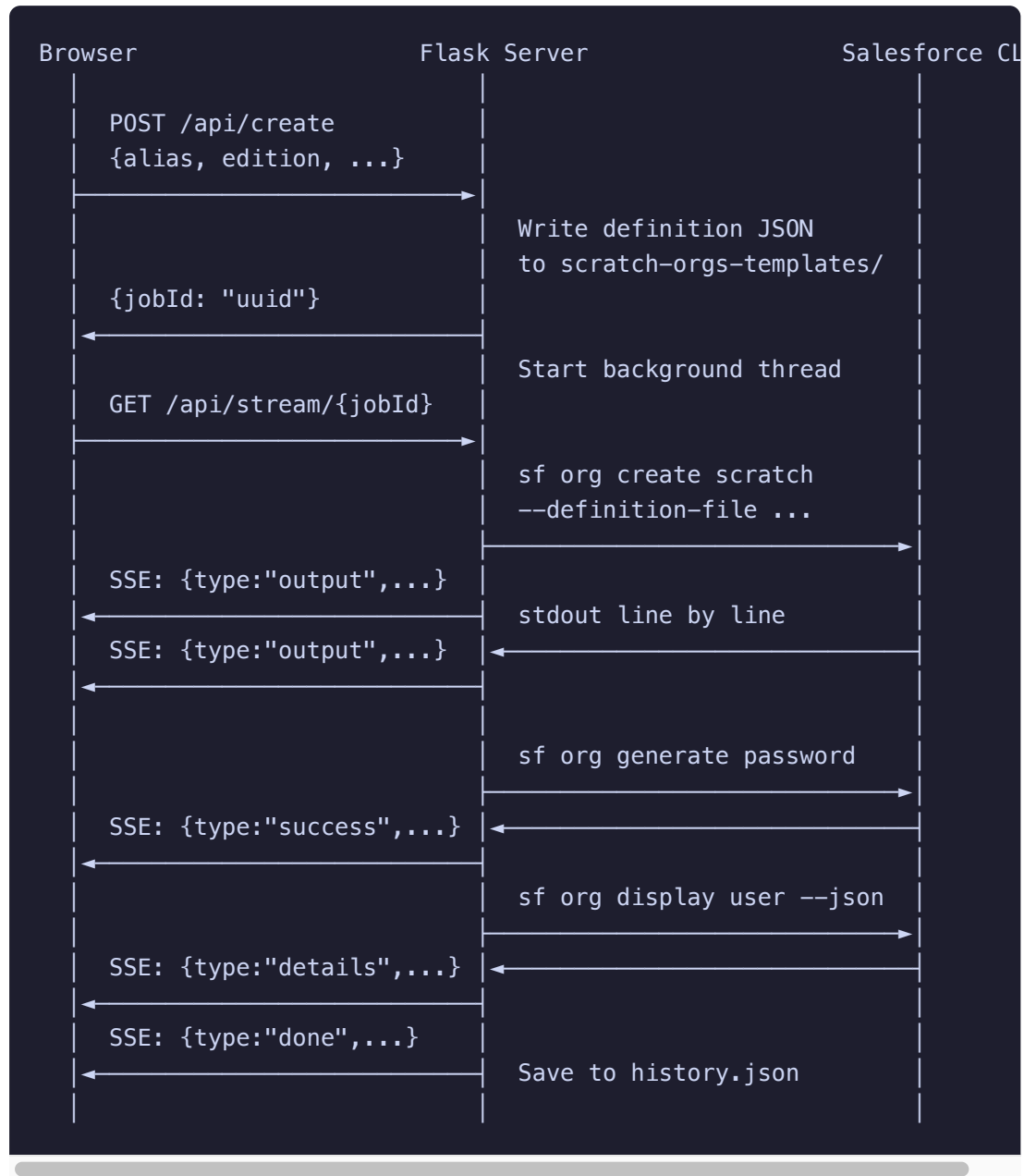
1. Check prerequisites
2. Skip extraction (version matches)
3. Skip Flask install (already in vendor/)
4. Start server, open browser

Installation directory: ~/.scratch-org-creator/

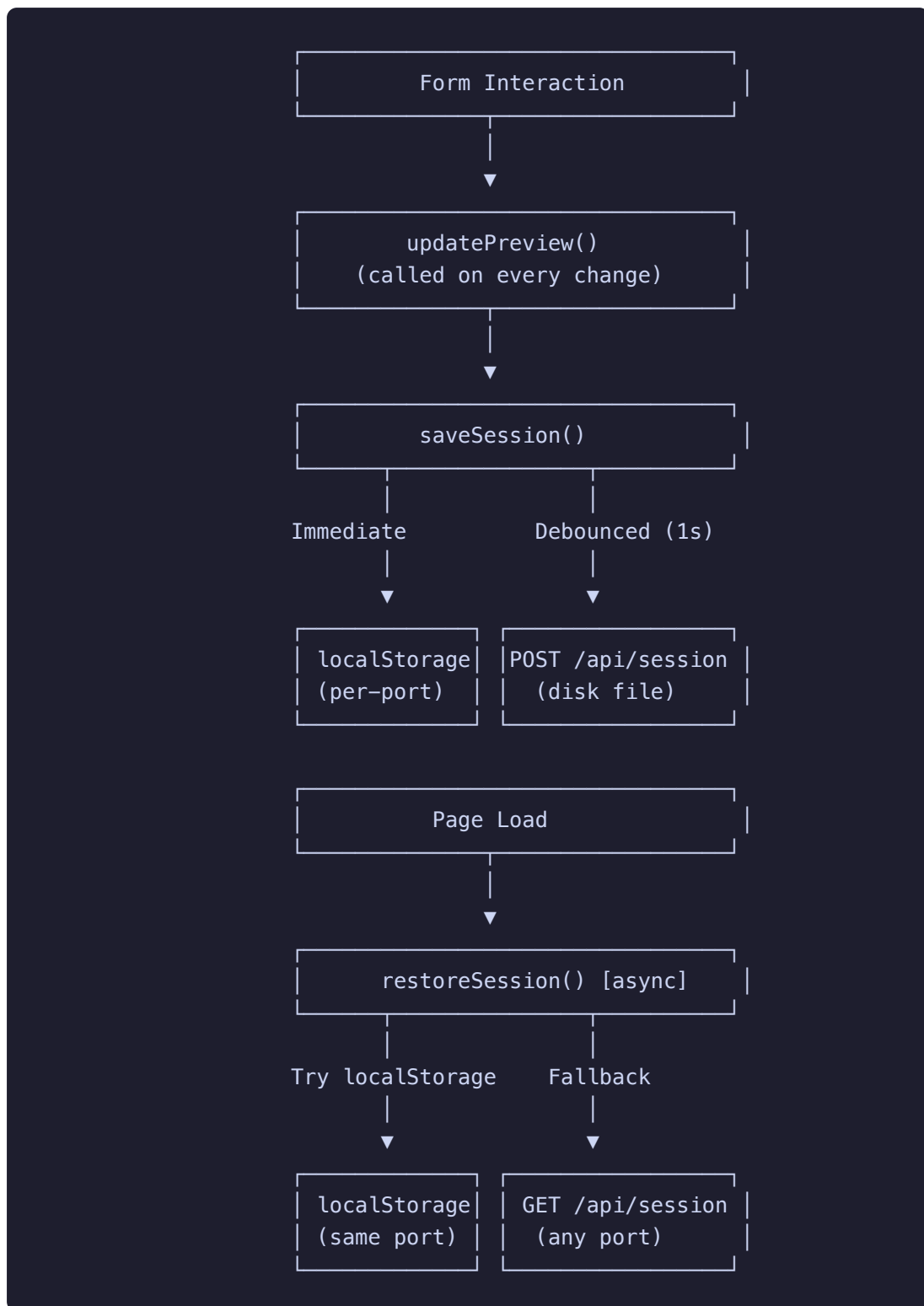
```
~/.scratch-org-creator/
├─ app.py
├─ features.json
├─ settings.json
├─ templates/
│   └─ index.html
├─ vendor/           (Flask + dependencies)
├─ .version          (content hash for update detection)
├─ session.json      (auto-created on use)
└─ history.json       (auto-created on use)
```

Data Flow

Scratch Org Creation Flow



Session Persistence Flow



API Endpoints

GET /

Returns the rendered `index.html` template.

GET /api/features

Returns the full features catalog as JSON array. Used by the frontend to populate the Choices.js multi-select dropdown.

Response: `[{name, description, category, requiresValue, valueRange?}, ...]`

GET /api/settings

Returns the settings catalog grouped by category.

Response: `[{category, settings: [{key, label, description, type, default}, ...]}, ...]`

GET /api/devhubs

Executes `sf org list --json` via subprocess. Parses the JSON output to find orgs marked as Dev Hubs (from `devHubs`, `nonScratchOrgs`, and `other` buckets where `isDevHub` is true).

Response: `[{alias, username}, ...]` **Timeout:** 30 seconds

POST /api/authorize

Executes `sf org login web` to initiate OAuth authorization flow. Opens a browser window for the user to log in.

Request body: `{alias: "DevHub", loginUrl: "https://login.salesforce.com"}` **Response:** `{success: true/false, alias?, message}` **Timeout:** 300 seconds (5 minutes for user to complete login)

GET /api/history

Returns the last 100 successful scratch org creations.

Response: [{alias, username, instanceUrl, edition, features, createdAt}, ...]

GET /api/session

Returns the saved form state from `session.json`, or `null` if no session exists.

POST /api/session

Saves the current form state to `session.json`. Called with 1-second debounce from the frontend.

DELETE /api/session

Removes `session.json`. Called after successful org creation.

POST /api/preview

Accepts form data and returns the scratch org definition JSON that would be written.

Request body: {orgName, edition, username?, country?, features?, settings?, ...} **Response:** The generated definition object

POST /api/create

Starts scratch org creation. If `_action: "open"` is in the body, runs `sf org open` instead.

Request body: {alias, orgName, edition, duration, devHub, features, settings, ...} **Response:** {jobId: "uuid"}

GET /api/stream/<job_id>

Server-Sent Events endpoint. Streams output from the running job.

Event format: data: {"type": "info|output|success|error|warning|details|done", "text": "..."}\n\n

POST `/api/shutdown`

Sends `SIGTERM` to the server process for graceful shutdown.

Frontend Architecture

5-Step Wizard

Step	Title	Key Interactions
1	Dev Hub	List authorized hubs, select one, or authorize new via OAuth
2	Definition	Alias, edition (with Partner warning), duration slider, username, country
3	Features	Choices.js multi-select with 296 features grouped by category, value inputs for numeric features
4	Settings	Accordion categories with toggle switches, search filter, custom dot-notation settings builder
5	Create	Review toggles, create button, real-time console output, org details card

Third-Party Libraries

Library	Version	Purpose	Loaded From
Choices.js	latest	Searchable multi-select dropdown	jsDelivr CDN

CSS Design System

```
:root {
  --primary: #0176d3;      /* Salesforce blue */
  --success: #2e844a;      /* Green for success states */
  --warning: #fe9339;      /* Orange for warnings */
  --error: #ea001e;        /* Red for errors */
  --bg: #f4f6f9;          /* Page background */
  --surface: #ffffff;      /* Card background */
  --text: #181818;         /* Primary text */
  --text-muted: #5a6872;   /* Secondary text */
  --border: #d8dde6;       /* Borders */
  --radius: 8px;          /* Border radius */
}
```

Toggle Switch Implementation

The toggle switch uses a CSS-only approach:

```
<label class="toggle">
  <input type="checkbox">
  <span class="toggle-slider"></span>
</label>
```

- `<label>` wraps everything so clicking anywhere toggles
 - `<input>` is hidden (`opacity: 0; width: 0; height: 0`)
 - `.toggle-slider` is positioned absolutely within the label
 - `.toggle-slider::before` is the white knob, animated with `transform: translateX(22px)`
 - `:checked + .toggle-slider` changes background from dark (`#1a1a1a`) to purple (`#7c3aed`)
 - `display: inline-block` on `.toggle` is **required** because `<label>` is inline by default and won't accept width/height without it
-

Session Persistence

Two-Layer Architecture

Layer 1: Browser localStorage (fast, per-origin) - Key:

`scratchOrgCreator_session` - Saved on every form change (synchronous) - Scoped to `http://localhost:PORT` — only works if port stays the same - The server uses preferred port 8484 to maximize localStorage hits

Layer 2: Server-side file (durable, port-independent) - File: `session.json` in the app directory - Saved with 1-second debounce via `POST /api/session` - Survives port changes, browser cache clears, different browsers - Loaded via `GET /api/session` as fallback when localStorage is empty

What's Saved

```
{
  "step": 3,
  "devHub": "MyDevHub",
  "alias": "my-scratch-org",
  "orgName": "My Scratch Org",
  "edition": "Enterprise",
  "duration": "14",
  "username": "admin@test.com",
  "country": "US",
  "description": "",
  "language": "en_US",
  "release": "",
  "hasSampleData": false,
  "features": ["Communities", "ServiceCloud"],
  "featureValues": {"FieldService": "5"},
  "toggledSettings": {
    "lightningExperienceSettings.enableS1DesktopEnabled": true,
    "chatterSettings.enableChatter": true
  },
  "customSettings": [
    {"key": "caseSettings.systemUserEmail", "val": "support@acme.com"}
  ],
  "setDefault": true,
  "generatePassword": true,
  "savedAt": "2026-05-26T00:15:00.000Z"
}
```

Restore Behavior

On page load: 1. `restoreSession()` checks `localStorage`, then `/api/session`
2. If session found with `step > 1`, navigates to that step 3. Shows a toast banner: "Session Restored — Your previous progress from Xm ago was restored"
4. Banner auto-dismisses after 10 seconds, or user clicks "Start Fresh" to clear

Clearing

Session is cleared (`localStorage.removeItem` + `DELETE /api/session`) when:
- Org creation completes successfully - User clicks "Start Fresh" on the restore banner

Distribution & Packaging

Development Mode

Run from the repository:

```
bash create-scratch-org.sh
```

Self-Contained Distribution

Generated by `python3 build-installer.py`:

```
# Give customer this single file:  
scratch-org-creator.sh (191 KB)
```

How the bundling works:

1. Each source file is base64-encoded
2. Each encoded payload is embedded in the shell script as a heredoc
3. On first run, each heredoc is piped through `python3 -c "import base64..."` to decode
4. Decoded files are written to `~/.scratch-org-creator/`
5. A `.version` file stores the content hash to skip extraction on subsequent runs

6. `--update` flag forces re-extraction

Update flow:

```
Developer edits source files
→ runs python3 build-installer.py
→ new scratch-org-creator.sh generated with new hash
→ sends to customer
→ customer runs it
→ hash differs from ~/.scratch-org-creator/.version
→ files re-extracted automatically
```

Security Considerations

Concern	Mitigation
Credentials in history.json	File is gitignored; stored locally only
Server accessible from network	Binds to <code>127.0.0.1</code> only (not <code>0.0.0.0</code>)
No authentication on endpoints	Local-only tool; not meant for shared/remote use
ANSI injection from CLI output	Regex strips all escape sequences before display
Shell injection	All CLI commands use array-based <code>subprocess</code> calls (no <code>shell=True</code>)
Vendor directory trust	Flask installed from PyPI via <code>pip3</code>

Dependencies

Runtime Dependencies

Dependency	Required By	How Provided
<code>python3</code> (3.8+)	Flask server, JSON handling	Must be pre-installed (ships with macOS)
<code>sf</code> CLI	All Salesforce operations	Must be pre-installed by user
Flask (Python)	Web server	Auto-installed to local <code>vendor/</code> directory
Choices.js	Feature multi-select	Loaded from jsDelivr CDN at runtime

Build Dependencies

Dependency	Required By	Notes
<code>python3</code>	<code>build-installer.py</code>	Standard library only (base64, hashlib, pathlib)

No Build Step Required

The application has no compile, transpile, or bundle step. HTML/CSS/JS are served as-is. The only "build" is generating the self-contained installer, which is optional.